SL 203 – Introduction to Algorithms and Software Programming

Indian Institute of Science

Course Project Report

Multimodal Pareto-Optimal Top-K Routing

Instructor	Dr. Nagarajan Natarajan
Students	Shreya Ghosh (25822)
	Soham Chakraborty (25845)
	Rahul Dev Sharma (26845)
Date	May 1, 2026

Centre for Infrastructure, Sustainable Transportation and Urban Planning

Indian Institute of Science (IISc), Bengaluru

Abstract

Multimodal transportation networks present a complex routing challenge due to the presence of multiple, often conflicting, optimization objectives such as travel time, monetary cost, number of transfers, and walking distance. Traditional single-objective shortest path algorithms fail to capture the trade-offs inherent in urban transit. This project provides an analysis of the Multimodal Pareto-Optimal Top-K Routing system, a label-setting algorithm designed to compute the Pareto frontier of non-dominated paths across a multi-layer graph integrating bus, metro, and walking segments. We detail the theoretical foundation, multi-criteria problem formulation, and the concrete implementation structure within the Python codebase. By mapping algorithmic concepts such as label propagation, Pareto dominance pruning, and diversity-aware Top-K selection, directly to the `urban_routing` source modules, this report serves as an exhaustive technical guide to the system's architecture, its algorithmic complexity, and its practical implementation details.

Contents

1 Introduction **4**
1.1 Problem Motivation . . . 4
1.2 The Need for Multi-Objective Optimization . . . 4
1.3 Real-World Relevance . . . 4

2 Background **4**
2.1 Pareto Optimality . . . 4
2.2 Multi-Criteria Shortest Path Problem (MCSPP) . . . 4
2.3 Label-Setting Algorithms . . . 5

3 Problem Formulation **5**
3.1 Graph Definition . . . 5
3.2 Cost Vector Definition . . . 5
3.3 Dominance and the Objective Function . . . 6

4 System Architecture **6**
4.1 Module Interaction . . . 6
4.2 Pipeline Diagram . . . 6

5 Algorithm Design **7**
5.1 Label Structure . . . 7
5.2 Label Propagation and Dominance Pruning . . . 7
5.3 Path Reconstruction . . . 8
5.4 Pseudocode . . . 8

6 Implementation Details **8**
6.1 Graph Representation: `core/graph.py` . . . 9
6.2 Labels and Frontiers: `core/label.py` . . . 9
6.3 Routing Engine: `algorithms/pareto_dijkstra.py` . . . 9
6.4 Diversity Selection: `selection/diversity_selector.py` . . . 9
6.5 Advanced Dominance Matrix: `algorithms/dominance.py` . . . 10

7 Complexity Analysis **10**
7.1 Theoretical Complexity . . . 10
7.2 Practical Considerations . . . 10
7.3 Bottlenecks . . . 10

8 Results and Experiments **11**
8.1 Experimental Setup . . . 11
8.2 Toy Network Analysis . . . 11
8.3 Urban Network Analysis . . . 14
8.4 Comparative Insights and Scaling Challenges . . . 14

9 Limitations **17**

10 Future Work **17**

11 Conclusion **17**

12 External Contribution: Pointer Visualization in C Code Visualizer **18**
12.1 Motivation 18
12.2 Target Project Overview 18
12.3 Contribution Description 18
12.4 Technical Implementation 18
12.5 Visual Demonstration 19
12.6 Learning Outcomes 20
12.7 Impact 20

13 Glossary **21**

1 Introduction

Urban public transportation networks are inherently multimodal, encompassing buses, metro lines, suburban rails, and walking segments. From a passenger's perspective, navigating these networks involves evaluating multiple conflicting criteria. A faster route might be significantly more expensive, while a cheaper route might involve multiple cumbersome transfers or excessive walking.

1.1 Problem Motivation

The traditional approach to routing relies on finding the shortest path with respect to a single metric (e.g., travel time) using algorithms like Dijkstra's or A*. However, single-objective optimization is inadequate for modern urban mobility. Commuters have diverse and subjective preferences; some prioritize time, others are budget-constrained, and some may wish to minimize physical exertion (walking distance) or carbon footprint. The Multimodal Pareto-Optimal Top-K Routing system addresses this challenge by generating a set of optimal trade-off paths, empowering the user to make an informed choice.

1.2 The Need for Multi-Objective Optimization

Multi-objective optimization in routing seeks to find a set of paths where no single path is strictly better than another across all criteria simultaneously. This set is known as the Pareto frontier. By computing the Pareto frontier, the system captures the full spectrum of optimal travel options. To prevent cognitive overload for the user, the system further filters this potentially large set of optimal paths into a concise "Top-K" list, ensuring diversity in the suggested routes.

1.3 Real-World Relevance

The codebase analyzed in this report (`urban_routing_project`) is designed to ingest real-world GTFS (General Transit Feed Specification) data for metro networks and route data for buses (e.g., BMTC). It constructs a realistic multi-layer graph with specific transfer penalties and realistic speed-estimated weights.

2 Background

Before detailing the system architecture, it is necessary to establish the theoretical background of multi-criteria routing and the specific algorithmic paradigms employed in this project.

2.1 Pareto Optimality

In a multi-objective optimization problem, a solution is considered Pareto-optimal if it cannot be improved in any of the objectives without degrading at least one other objective. Given two cost vectors $\mathbf{u} = (u_1, \ldots, u_d)$ and $\mathbf{v} = (v_1, \ldots, v_d)$ where all objectives are to be minimized, we say that $\mathbf{u}$ *dominates* $\mathbf{v}$ (denoted $\mathbf{u} \prec \mathbf{v}$) if and only if:

$$\forall i \in \{1, \ldots, d\}, u_i \leq v_i \quad \text{and} \quad \exists j \in \{1, \ldots, d\}, u_j < v_j \tag{1}$$

The set of all non-dominated solutions forms the Pareto frontier.

2.2 Multi-Criteria Shortest Path Problem (MCSPP)

The MCSPP is an extension of the standard shortest path problem where each edge in the graph is assigned a vector of weights instead of a single scalar value. Finding the complete

Pareto frontier for MCSPP is known to be NP-hard in the general case, as the number of Pareto-optimal paths can grow exponentially with the size of the graph [1, 3].

2.3 Label-Setting Algorithms

To solve the MCSPP, algorithms typically generalize Dijkstra's algorithm. These are known as *label-setting* or *label-correcting* algorithms [2]. In the label-setting variant (such as Martins' algorithm), each node maintains a set (frontier) of non-dominated labels. A label represents a partial path from the origin to that node, characterized by its accumulated cost vector. During the algorithm's execution, the label with the lexicographically smallest or minimum primary objective cost is extracted from a priority queue (the "open set"), and it is propagated to its neighboring nodes. A new label generated at a neighbor is only kept if it is not dominated by any existing label at that neighbor.

3 Problem Formulation

The transportation network is modeled as a directed, multi-layer multigraph.

3.1 Graph Definition

Let the multimodal network be defined as $G = (V, E)$, where:

- V is the set of nodes, representing physical stops (bus stops, metro stations).
- E is the set of directed edges, where an edge $e = (u, v)$ connects node u to node v.

Since there can be multiple routes connecting the same two stops (e.g., two different bus routes, or a bus route and a walk segment), the graph is a *multigraph*. Each edge $e \in E$ is associated with a specific transportation mode $M_e \in \{\text{Bus}, \text{Metro}, \text{Walk}\}$.

3.2 Cost Vector Definition

Each edge $e \in E$ has an associated multidimensional weight vector representing the cost of traversing that edge. Based on the `EdgeWeight` dataclass defined in `data/schema.py`, the cost vector $\mathbf{w}(e)$ is defined as a 5-tuple:

$$\mathbf{w}(e) = (t_e, c_e, x_e, w_e, g_e) \tag{2}$$

where:

- t_e: Travel time in minutes.
- c_e: Monetary cost in INR.
- x_e: Number of transfers (incurred when switching modes or routes).
- w_e: Walking distance in meters.
- g_e: Carbon footprint (CO_2 emissions) in grams.

The total cost of a path $P = (e_1, e_2, \ldots, e_k)$ is the element-wise sum of the edge weights along the path:

$$\mathbf{W}(P) = \sum_{i=1}^{k} \mathbf{w}(e_i) \tag{3}$$

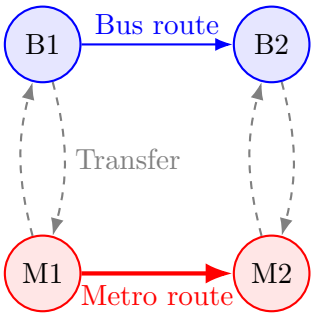

Figure 1: Example of a Multimodal Graph with Bus, Metro, and Walk edges.

3.3 Dominance and the Objective Function

The objective of the routing algorithm is to find a set of paths $\mathcal{P}_{s,t}$ from a source node s to a target node t such that no path in $\mathcal{P}_{s,t}$ is dominated by another path. Mathematically, a path P_1 dominates P_2 ($P_1 \prec P_2$) if:

$$\mathbf{W}(P_1) \leq \mathbf{W}(P_2) \quad \text{and} \quad \mathbf{W}(P_1) \neq \mathbf{W}(P_2) \tag{4}$$

The algorithm outputs $\mathcal{P}^* \subseteq \mathcal{P}_{s,t}$, which is the Pareto frontier of paths.

4 System Architecture

The system is designed with a highly modular architecture, clearly separating data ingestion, graph representation, algorithmic logic, and solution selection.

4.1 Module Interaction

The system comprises the following core modules:

1. `data/`: Handles loading and parsing of input data. The `loader.py` parses GTFS (metro) and proprietary bus data. The `fuser.py` creates intermodal transfer links (e.g., connecting a metro station to a nearby bus stop). The `schema.py` defines the fundamental data types (`Stop`, `Route`, `EdgeWeight`).

2. `core/`: Contains the graph data structure and the label definition. `graph.py` builds the `MultimodalGraph` by layering bus edges, metro segments, and walking links. `label.py` defines the `Label` class representing partial paths and the `LabelFrontier` for managing non-dominated labels at each node.

3. `algorithms/`: Houses the core routing logic. `pareto_dijkstra.py` implements the multi-criteria label-setting algorithm. `dominance.py` provides efficient mathematical routines for Pareto dominance checks.

4. `selection/`: After the Pareto frontier is generated, it often contains too many paths. `diversity_selector.py` implements a greedy algorithm using Jaccard dissimilarity to extract a diverse Top-K subset of paths.

4.2 Pipeline Diagram

The following figure illustrates the data flow and system pipeline.

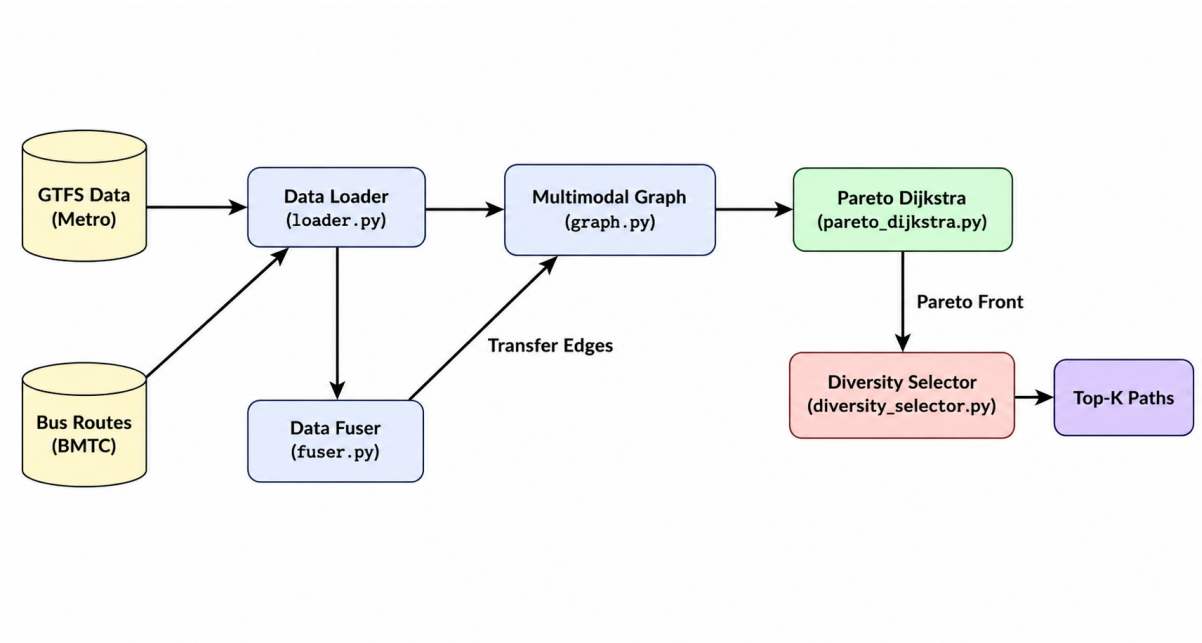

Figure 2: System Architecture and Data Pipeline.

5 Algorithm Design

The routing engine relies on a multi-criteria variant of Dijkstra's algorithm, maintaining sets of non-dominated labels instead of scalar values.

5.1 Label Structure

A *Label* at a node v represents a discovered path from the origin to v. As defined in `core/label.py`, it encapsulates:

- `cost`: The multidimensional `EdgeWeight` accumulated so far.
- `node`: The current node v.
- `prev`: A pointer to the predecessor label, enabling path reconstruction.
- `edge`: The specific `GraphEdge` traversed to reach v.

5.2 Label Propagation and Dominance Pruning

The algorithm maintains a priority queue (min-heap) of active labels, prioritized by the primary objective (travel time, t_e). At each node v, a `LabelFrontier` maintains a list of locally non-dominated labels. When a new label L_{new} arrives at node v:

1. It is compared against all existing labels in v's frontier.
2. If L_{new} is dominated by *any* existing label, it is discarded.
3. If L_{new} is not dominated, it is added to the frontier. Any existing labels that are now dominated by L_{new} are removed.
4. If added, L_{new} is pushed into the global priority queue for future expansion.

To prevent exponential explosion in dense networks, the frontier size per node is strictly capped by a configurable parameter (`MAX_LABELS_PER_NODE`).

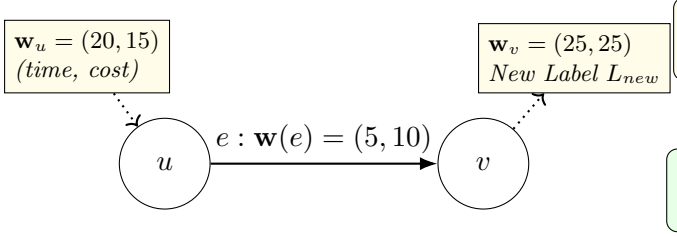

Figure 3: Label Propagation Process.

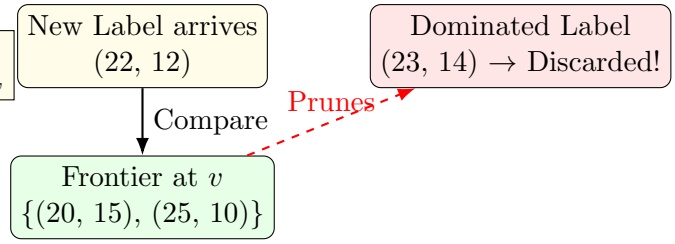

Figure 4: Dominance Pruning Check.

5.3 Path Reconstruction

Once the priority queue is empty or an early termination condition is met (e.g., when the primary objective of the popped label far exceeds the best known time at the destination), the algorithm terminates. For every label remaining in the destination node's frontier, the system walks backward through the `prev` pointers to reconstruct the exact sequence of nodes and edges, forming a `ParetoPath`.

5.4 Pseudocode

The multi-criteria label-setting algorithm is formalized in Algorithm 1.

Algorithm 1 Multi-Criteria Pareto Dijkstra

Require: Graph $G = (V, E)$, Origin s, Destination t
Ensure: List of Pareto-optimal paths to t

```
Initialize Q as an empty priority queue (min-heap on time)
Initialize F[v] ← ∅ for all v ∈ V                                  ▷ Frontiers
L_start ← Label(cost = 0, node = s, prev = null)
F[s].add(L_start)
Q.push(L_start)
while Q is not empty do
    L ← Q.pop()
    if early_termination_condition(L, F[t]) then
        continue
    end if
    for each edge e = (L.node, v) in E do
        w_new ← L.cost + w(e)
        L_new ← Label(cost = w_new, node = v, prev = L, edge = e)
        if F[v].try_add(L_new) then                                ▷ Dominance check inside
            Q.push(L_new)
        end if
    end for
end while
return {reconstruct_path(L) for L in F[t]}
```

6 Implementation Details

The theoretical concepts are meticulously implemented in Python. This section maps the algorithmic components directly to the analyzed `urban_routing` codebase.

6.1 Graph Representation: `core/graph.py`

The network is modeled using NetworkX's `MultiDiGraph` within the `MultimodalGraph` class. The graph is constructed in "layers":

1. **Bus Edges:** Iterates over BMTC routes, calculating distance using the Haversine formula and estimating weights via `mode_edge_weight()`.
2. **Metro Edges:** Ingests GTFS segments with deterministic travel times.
3. **Transfer Edges:** Adds edges connecting different modes (e.g., bus stop to metro station).
4. **Walk Edges:** In lieu of full OSM integration, `_add_walk_edges()` implements a spatial sweep-line approach (sorting stops by latitude) to add straight-line walking edges between nearby stops ($\leq$ MAX_WALK_TRANSFER_M), efficiently avoiding an $O(n^2)$ spatial comparison.

6.2 Labels and Frontiers: `core/label.py`

The `Label` class utilizes Python's `@dataclass`. The critical Pareto dominance logic is implemented in the `dominates` method:

```python
def dominates(self, other: "Label") -> bool:
    a = self.cost.as_tuple()
    b = other.cost.as_tuple()
    at_least_as_good = all(x <= y for x, y in zip(a, b))
    strictly_better  = any(x <  y for x, y in zip(a, b))
    return at_least_as_good and strictly_better
```

Listing 1: Dominance check in `core/label.py`

The `LabelFrontier` class wraps a list of labels. Its `try_add(new_label)` method scans existing labels to determine if `new_label` is dominated. If it survives, it removes any existing labels that `new_label` dominates and enforces the `MAX_LABELS_PER_NODE` cap.

6.3 Routing Engine: `algorithms/pareto_dijkstra.py`

The `ParetoDijkstra` class encapsulates the pathfinding logic. It uses Python's `heapq` for the open set. Because `Label` objects might have identical primary priorities, a monotonically increasing `counter` integer is pushed into the heap tuples `(priority, counter, label)` to deterministically break ties and prevent the heap from attempting to evaluate the `<` operator directly on complex label objects. An early termination optimization is included:

```python
if current.cost.time_min > best_dest_time * 3:
    continue
```

This pruning assumes that a route taking three times longer than the fastest known route will not yield a practically useful Pareto-optimal alternative, saving significant computation time.

6.4 Diversity Selection: `selection/diversity_selector.py`

Given that the Pareto frontier can contain dozens of slightly varied paths, the `DiversitySelector` reduces this to K paths. It operates in two spaces:

- **Objective Space:** Cost vectors are normalized to $[0, 1]$. The algorithm greedily selects paths that maximize the L_2 distance to already selected paths in this space (*farthest-point sampling*).

- **Structural Space:** It enforces a structural difference constraint using *Jaccard Dissimilarity* on the set of edges comprising the paths:

$$d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} \tag{5}$$

  A path is only added to the Top-K set if its Jaccard dissimilarity with all previously selected paths exceeds a configured threshold, ensuring routes physically differ from one another.

6.5 Advanced Dominance Matrix: `algorithms/dominance.py`

For post-processing and metric calculation, `dominance.py` provides vectorized Numpy implementations. The `fast_pareto_filter` function performs Pareto filtering over matrices of shape (n, d), heavily utilizing Numpy broadcasting for computational efficiency. It also implements algorithms for calculating Hypervolume (both 2D exact and N-D Monte Carlo approximations) to measure the quality of the generated Pareto fronts.

7 Complexity Analysis

7.1 Theoretical Complexity

Let $|V|$ be the number of nodes, $|E|$ the number of edges, and d the number of objectives. Let F_{max} denote the maximum number of non-dominated labels maintained at any node (enforced as `MAX_LABELS_PER_NODE`). In the worst case, every edge expansion involves comparing the new label against all existing labels in the destination node's frontier. The cost of a single dominance check in d dimensions is $O(d)$. Thus, attempting to add a label takes $O(F_{max} \cdot d)$ time. Since each label popped from the priority queue can trigger expansions along all outgoing edges, the maximum number of heap insertions is bounded by $O(|E| \cdot F_{max})$. The overall theoretical time complexity is therefore bounded by:

$$O(|E| \cdot F_{max} \cdot (d \cdot F_{max} + \log(|V| \cdot F_{max}))) \tag{6}$$

The space complexity is dominated by the storage of labels, which requires $O(|V| \cdot F_{max})$ memory.

7.2 Practical Considerations

In practice, F_{max} is kept relatively small (e.g., 50-100) via configuration. Furthermore, urban networks exhibit strong spatial locality, and the early termination condition truncates paths that wildly exceed the shortest travel time. As a result, the average frontier size per node is significantly less than F_{max}, and the actual runtime is acceptable for query-based online routing.

7.3 Bottlenecks

Profiling the codebase reveals two primary computational bottlenecks:

1. **Frontier Maintenance**: The inner loop `try_add()` iterates over lists in Python. For large values of F_{max}, Python's interpreter overhead in executing the list comprehensions and tuple comparisons within `dominates()` can become significant.

2. **Graph Construction (Walk Edges)**: Although `_add_walk_edges()` utilizes a sorted latitude sweep, constructing walking links in a dense graph with thousands of stops requires $O(n \log n + n \cdot c)$ distance calculations.

8 Results and Experiments

8.1 Experimental Setup

To validate the Multimodal Pareto-Optimal Top-K Routing system, experiments were conducted on two distinct environments: a controlled toy network and a real-world urban routing network. The **Toy Network** (`toy_multimodal_pareto`) serves as an intuitive testbed. It features a grid road network, straight metro lines, and walking links between nearby nodes. This synthetic environment provides controlled trade-offs and repeatable experiments, allowing us to explicitly observe the mechanics of label expansion and dominance pruning. The **Urban Network** (`urban_routing_project`) scales the system to real-world complexity, ingesting actual GTFS data for metro operations and proprietary transit data for buses, challenging the algorithm with dense topologies and realistic mode-dependent costs.

8.2 Toy Network Analysis

The synthetic network establishes the baseline behavior of the multi-objective label-setting algorithm.

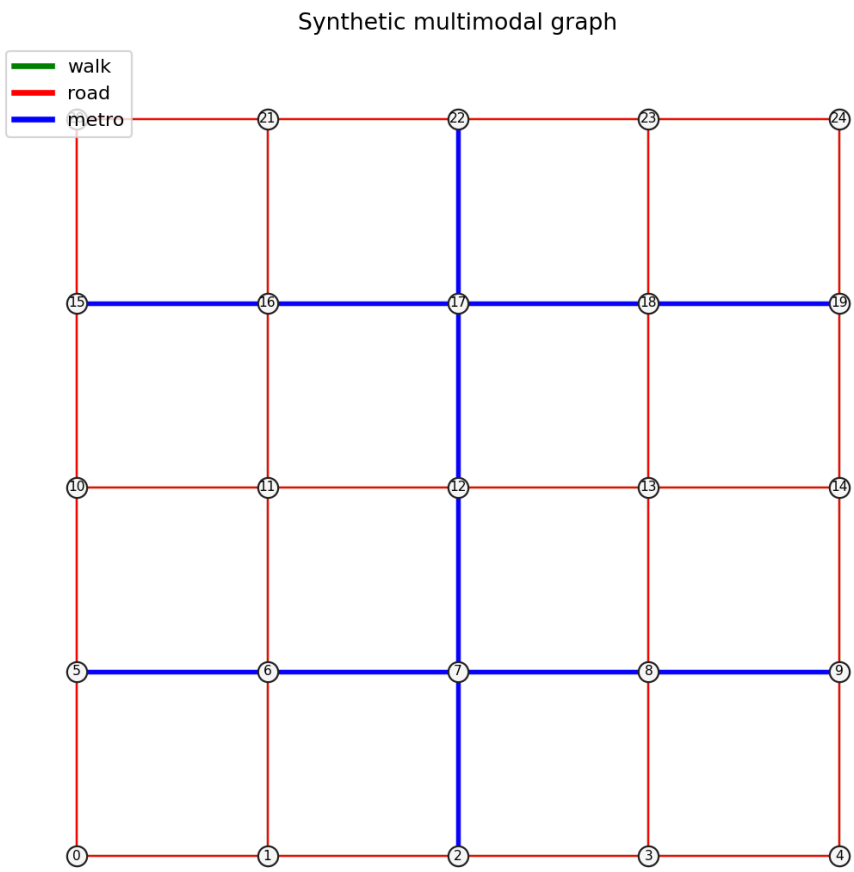

Figure 5: The synthetic multimodal graph consisting of a grid road network (red), parallel metro lines (blue), and straight-line walking transfers (green dashed lines). Nodes S and T denote the source and target respectively.

As shown in Figure 5, the network provides multiple valid paths from source S to target T. Without Pareto dominance, a standard Dijkstra algorithm would collapse these objectives into

a single weighted score, keeping only one path. Instead, the multi-objective approach generates a full frontier.

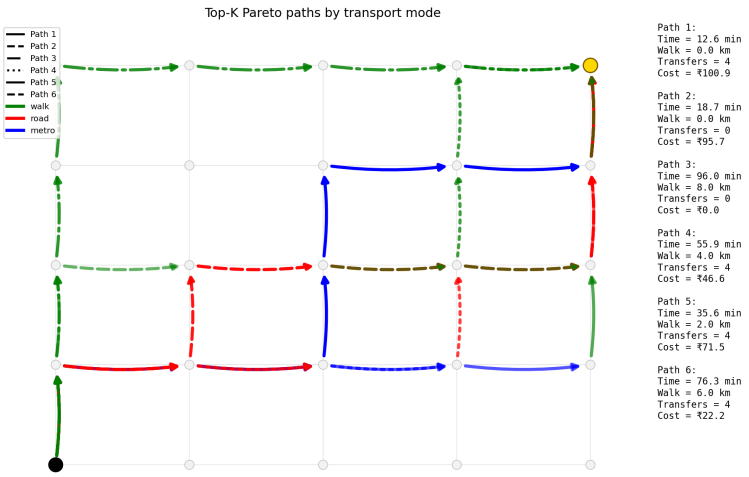

Figure 6: All selected Pareto paths overlaid on the toy network. Each segment is color-coded by transportation mode (red for road, blue for metro, green for walking).

Figure 6 visualizes the Top-K diverse paths selected from the full Pareto frontier. The diversity-based selection ensures that paths spanning the entire objective space—fast but expensive, slow but free, and balanced trade-offs—are presented to the user.

To understand how dominance pruning filters these paths, consider the individual path profiles:

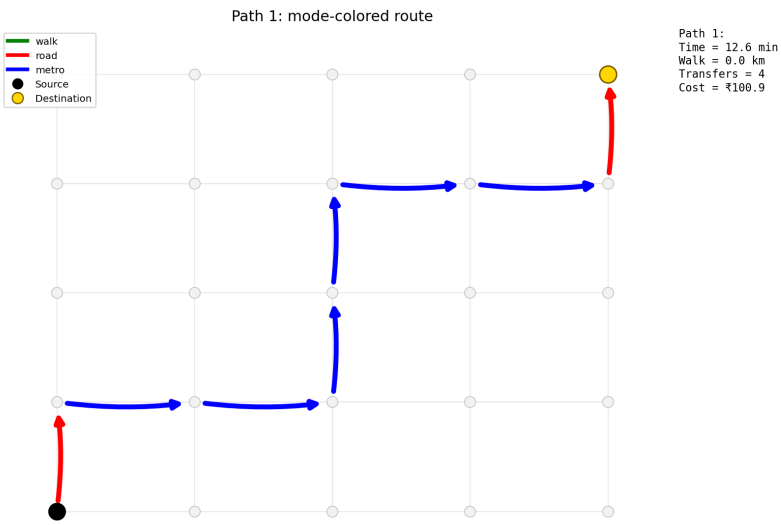

Figure 7: Path 1: The fastest representative path. It relies heavily on rapid transit (metro) and road segments but incurs high monetary cost and multiple transfers.

As seen in Figures 7 and 8, Path 1 and Path 3 are incomparable. Path 1 dominates in travel time, while Path 3 dominates in monetary cost and transfers. The algorithm correctly retains both. However, any partial route that takes as much time as Path 3 but costs more than Rs. 0 would be strictly dominated and pruned from the respective node's frontier, preventing label explosion.

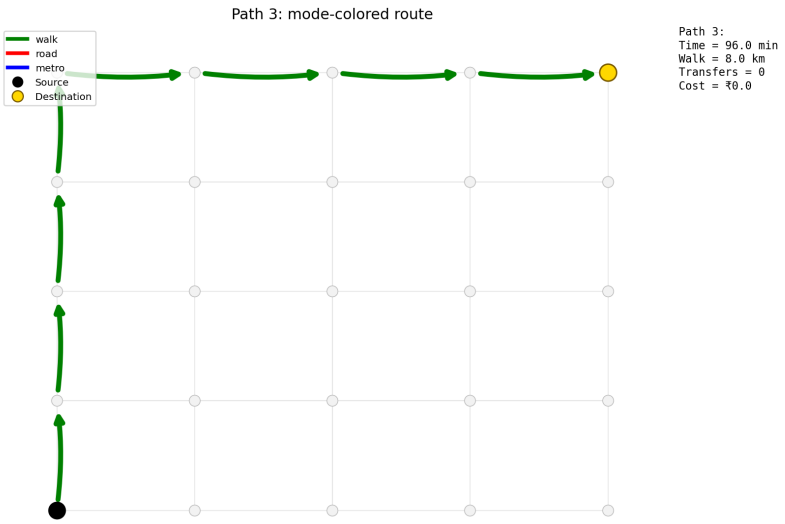

Figure 8: Path 3: A walking-only path. It offers zero monetary cost and zero transfers but demands significantly higher travel time and physical effort.

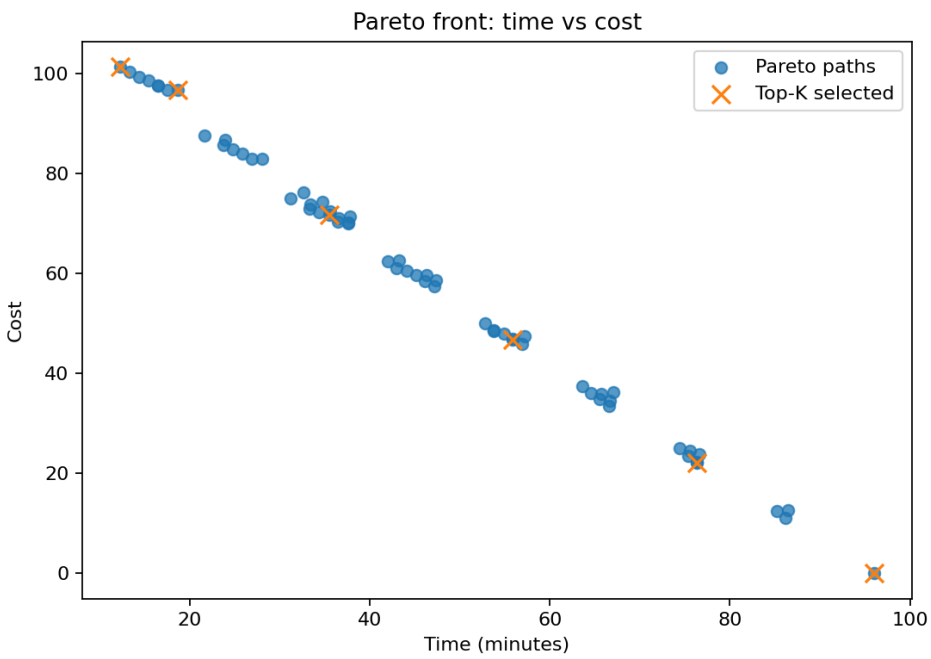

Figure 9: Computed Pareto frontier in the 2D time-cost subspace for the toy network. The non-dominated points form a distinct convex hull, illustrating the direct trade-off between travel speed and affordability.

Figure 9 plots the resulting frontier in the time-cost plane. The visualization explains why the Pareto set has structure: fast paths have high costs, cheap paths have high times, and balanced paths occupy the "knee" of the curve.

8.3 Urban Network Analysis

Transitioning to the real-world urban routing dataset demonstrates the algorithm's scalability.

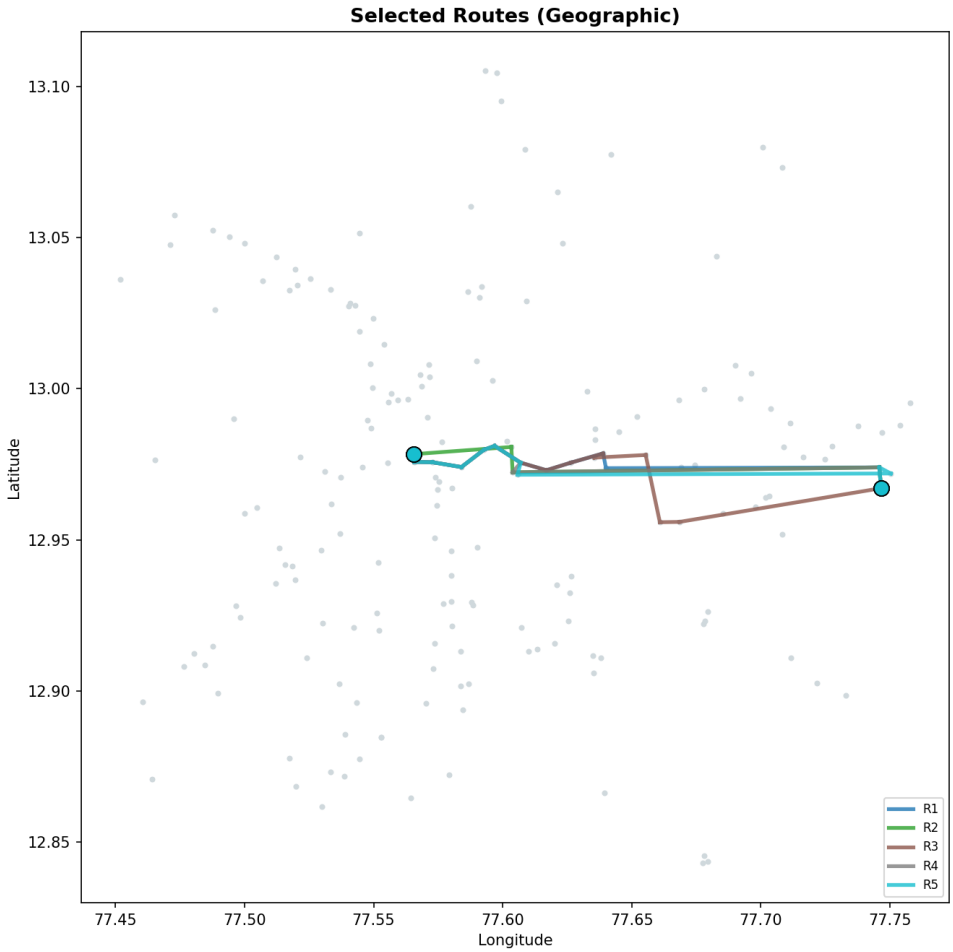

Figure 10: Interactive route map visualization of the selected Pareto-optimal paths in the real urban network. The topological complexity is vastly greater than the toy model, featuring irregular bus routes and constrained metro corridors.

In the urban network (Figure 10), the algorithm navigates realistic constraints such as fixed metro stations and meandering bus routes. The diversity selection becomes critical here, as the full Pareto set often contains hundreds of slightly varied paths (e.g., getting off a bus one stop earlier and walking).

Figure 12 and Figure 13 illustrate the multidimensional complexity of the urban results. The parallel coordinates and radar charts explicitly show that a single path rarely minimizes more than one objective simultaneously. The greedy max-min Jaccard selection successfully extracts distinct, actionable choices from the dense frontier shown in Figure 11.

8.4 Comparative Insights and Scaling Challenges

Comparing the toy and urban analyses highlights fundamental scaling challenges. While the toy network produces a highly structured and easily interpretable Pareto set, the urban network's

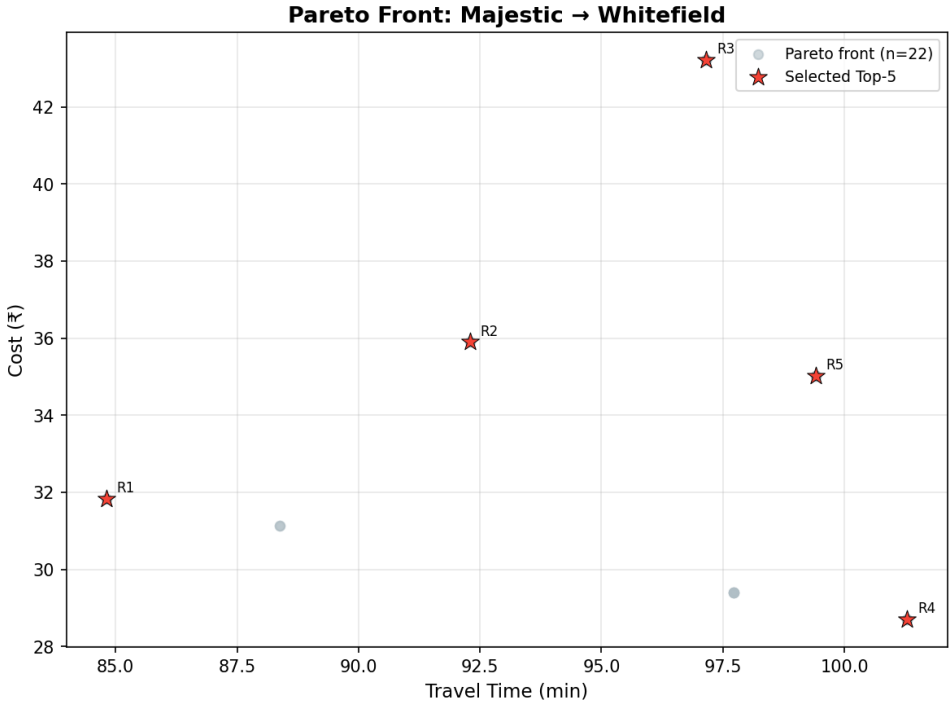

Figure 11: Pareto frontier in time-cost space for the urban network. The dense cluster of points reflects the high combinatorial variety of real-world transit options.

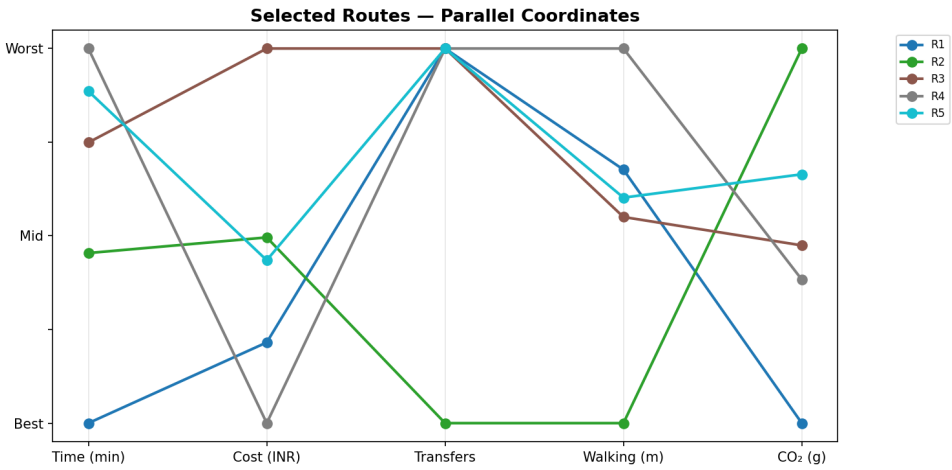

Figure 12: Parallel coordinates plot visualizing the 4-dimensional objective vectors (Time, Cost, Transfers, Walking) of the selected Top-K paths in the urban environment.

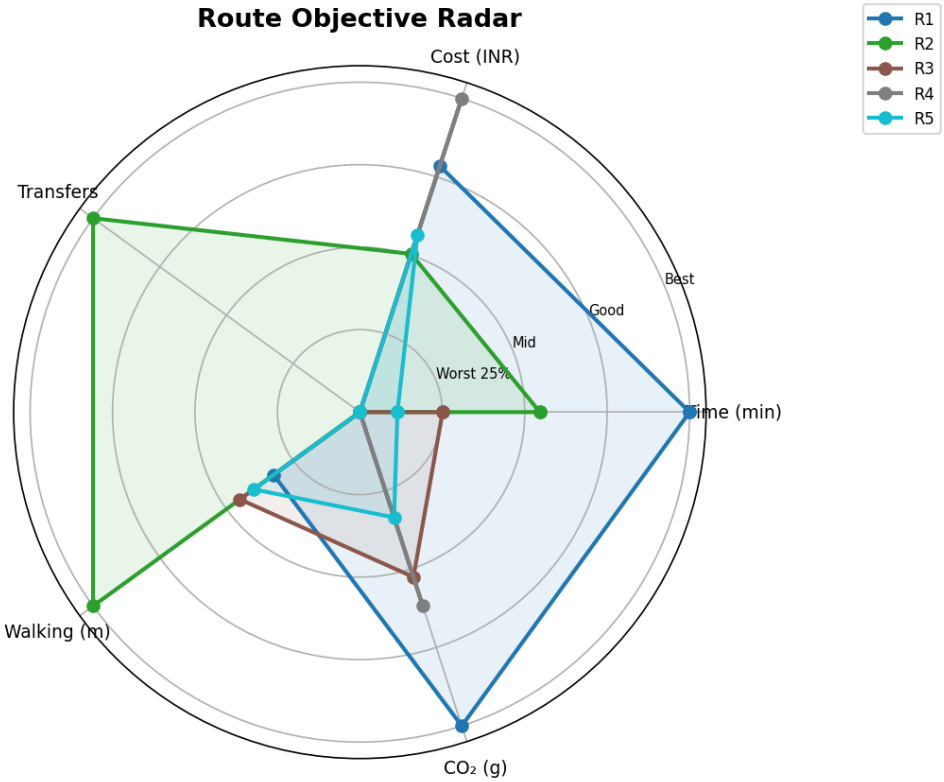

Figure 13: Radar (spider) chart highlighting the normalized multi-objective trade-offs of the final suggested urban routes. It clearly delineates the 'Fastest', 'Cheapest', and 'Eco-Friendly' profiles.

frontier grows rapidly due to minor variations in physical geometry. In the real network, enforcing the Jaccard structural dissimilarity constraint during Top-K selection is vital; without it, the Top-K paths would simply be minor geometric variations of the absolute fastest path. Furthermore, managing the maximum label size (F_{max}) is necessary in the urban context to prevent exponential runtime growth, a limitation that is barely noticeable in the controlled toy environment.

9 Limitations

Despite its robustness, the current architecture has a few limitations:

1. **Static Timetables:** The system assumes static frequency-based bus headways and fixed GTFS segments. It does not account for real-time traffic congestion or dynamic delays.
2. **Performance in Python:** Because the core expansion loop executes purely in Python, the algorithm is slower than heavily optimized C++ engines (like OSRM or Valhalla).
3. **Walk Generation Strategy:** The heuristic of generating walk edges via straight-line Haversine distance ignores physical barriers (e.g., rivers, highways), occasionally suggesting impossible transfers.

10 Future Work

Future enhancements to the `urban_routing` codebase could focus on:

- **A* Heuristics:** Implementing multi-objective A* (MOA*) by utilizing geographic distance as an admissible heuristic to guide the search towards the destination, significantly reducing the expanded search space.
- **Cython/Rust Acceleration:** Moving the `LabelFrontier` and `try_add` logic into a compiled extension to bypass Python overhead.
- **Integration with OpenStreetMap (OSM):** Replacing straight-line walk edges with actual pedestrian routing data to improve the realism of intermodal transfers.

11 Conclusion

This report has detailed the design and implementation of a Multimodal Pareto-Optimal Routing system. By reformulating urban navigation as a multi-criteria label-setting problem, the system successfully extracts optimal trade-off paths considering time, cost, transfers, and environmental impact. The direct mapping of theoretical constructs like Pareto dominance and Jaccard-based diversity selection to the Python modules demonstrates a highly cohesive and sound software architecture. While computational bottlenecks exist due to the language choice, the underlying algorithms are theoretically rigorous and yield highly practical routing intelligence.

12 External Contribution: Pointer Visualization in C Code Visualizer

12.1 Motivation

A critical component of this coursework involves extending technical collaboration beyond the confines of our primary project. Engaging in open-source contributions provides exposure to real-world codebases, version control workflows, and collaborative problem-solving. By contributing to an external repository, we aimed to bridge theoretical learning with practical software engineering, emphasizing educational tool development.

12.2 Target Project Overview

The target repository, **C Code Visualizer** (`https://github.com/Kuldeep1709/C-Code-Visualizer`), is an educational platform designed to visualize Data Structures and Algorithms (DSA) and fundamental program execution dynamics. It serves as an interactive debugging and learning environment where students can step through C code and observe how memory, variables, and data structures mutate in real-time.

12.3 Contribution Description

This contribution involved implementing a **Pointer Visualization** feature (`https://github.com/Kuldeep1709/C-Code-Visualizer/pull/1`), targeting the `pointer-visualization` branch.

Specifically, the feature:

- Explicitly shows pointer variables and their assigned memory addresses.
- Visualizes the dereferencing operation (`*ptr`), drawing an arrow-style mapping between the pointer and the memory cell it points to.
- Tracks value changes during step-by-step execution, dynamically updating the pointer's target value in the user interface.

This heavily improves the educational value of the project. Pointers are notoriously difficult for beginners to grasp conceptually. By visually coupling the memory address with the target value, users can immediately observe the impact of pointer arithmetic and dereferencing, making debugging significantly more intuitive.

12.4 Technical Implementation

The implementation bridges the backend execution engine with the frontend UI framework (built with TypeScript and Vite).

- **Backend State Tracking**: The C++ debugging engine was modified to preserve pointer metadata within the backend debug state, preventing memory addresses from being discarded during execution steps.
- **Frontend Variable Panel**: The UI was updated to render pointer variables differently from primitive types. The variables panel now parses the pointer metadata and constructs an arrow-style visual link.
- **Execution Mapping**: During step-by-step debugging, the UI polls the active state. If a pointer dereference occurs, the interface immediately highlights both the pointer variable and the specific memory block it targets, updating the `*ptr` value dynamically.

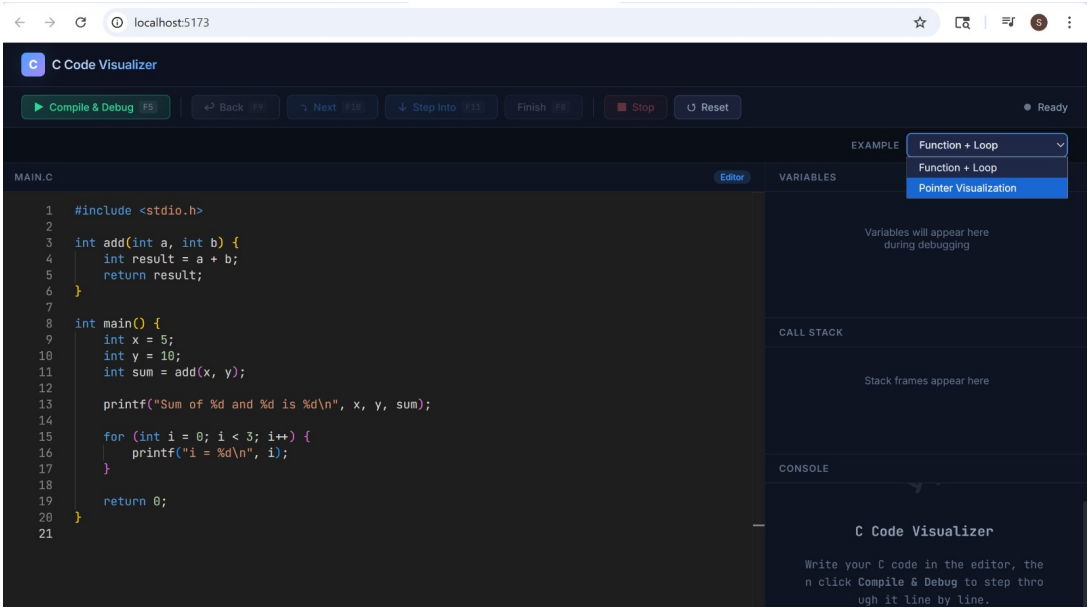

Figure 14: Added example selector dropdown with a new Pointer Visualization demo program.

12.5 Visual Demonstration

Figure 14 demonstrates the initial state where a pointer is declared and assigned. The variable panel captures the specific memory address and displays it alongside the variable name, directly mirroring standard C semantics.

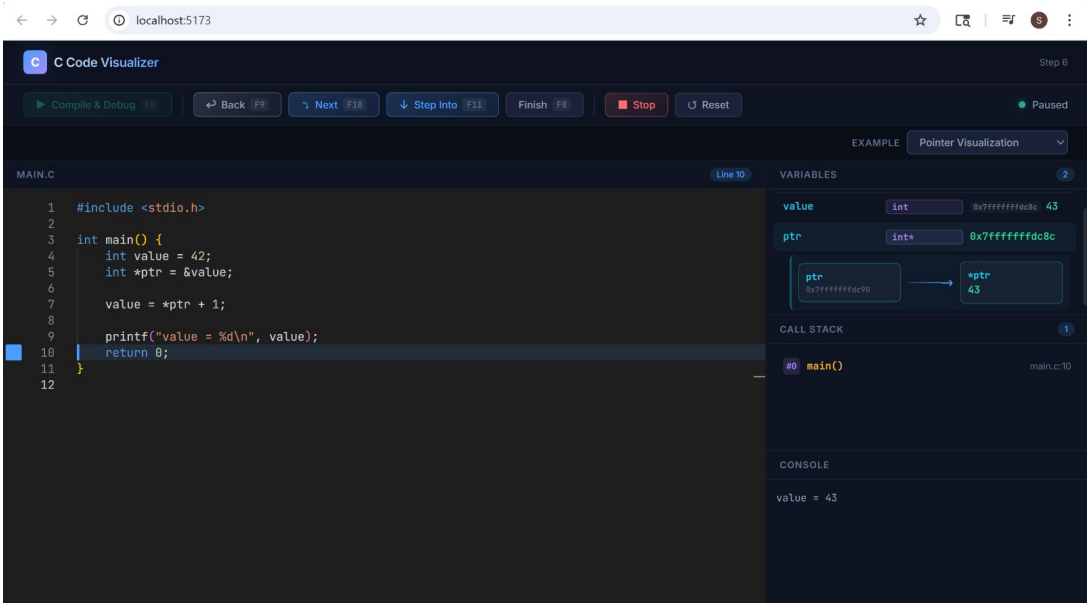

Figure 15: Step-by-step execution visualizing the dereferencing operation (`*ptr`). The value of the dereferenced pointer is evaluated and displayed.

As the program steps forward (Figure 15), the visualization explicitly shows the dereferencing step. The interface draws attention to the fact that reading or modifying `*ptr` interacts directly with the underlying target variable.

Figure 16 shows the final tracked state. The visual connection remains persistent, ensuring that the user understands the indirect memory access paradigm throughout the program's lifecycle.

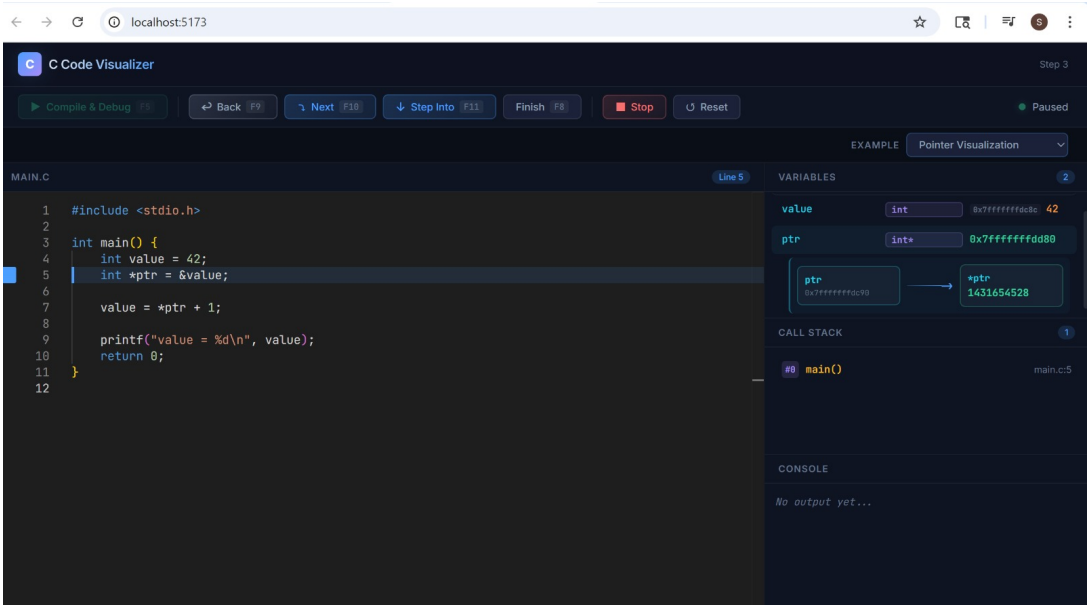

Figure 16: Final output state showing tracked value changes. The UI continues to link the pointer to its target even as other variables are updated.

12.6 Learning Outcomes

Contributing to this project yielded substantial educational benefits:

- **Deeper Systems Understanding**: Modifying a C++ debugging backend to correctly extract and format pointer metadata required a deep understanding of memory layout and stack frame inspection.
- **Full-Stack Integration**: The PR necessitated cross-boundary development, ensuring that the C++ backend emitted JSON state compatible with the TypeScript/Vite frontend.
- **Real-World Collaboration**: Operating within a foreign codebase reinforced best practices regarding pull request workflows, documentation, and modular feature integration.

12.7 Impact

The addition of pointer visualization transforms the C Code Visualizer from a basic variable tracker into a robust systems-level educational tool. By demystifying memory indirection, it significantly lowers the barrier to entry for students learning C. Future extensions could build upon this foundation to visualize complex linked data structures (like graphs and trees) and dynamic memory allocation (`malloc`/`free`), directly applying these UI mechanics.

13 Glossary

Routing Project Terms

Term	Definition
Pareto Optimality	A state of resource allocation where it is impossible to improve one objective without simultaneously degrading at least one other objective.
Dominance	In multi-objective routing, a path dominates another if it is strictly better in at least one metric and no worse in any other metric.
Label	A mathematical tuple attached to a node that records the accumulated cost metrics (Time, Cost, Transfers, Walk) of a specific path from the origin.
Frontier	The collection of all non-dominated labels (Pareto-optimal paths) available at a given node or for a specific origin-destination pair.
Multimodal Graph	A transportation network model integrating distinct modes of transit (e.g., road, metro, walking) into a single cohesive directed graph.
Cost Vector	An n-dimensional vector storing the specific values for each optimization objective (e.g., $[time, cost, transfers, walk_dist]$) for a given edge or path.
Top-K Paths	A filtered subset of the Pareto frontier consisting of the K most structurally diverse and representative routes.

Systems and Algorithm Terms

Term	Definition
Priority Queue	A fundamental data structure where each element has a "priority" associated with it. Elements with higher priority are served before those with lower priority (used heavily in Dijkstra's algorithm).
Dijkstra Algorithm	A classic algorithm for finding the shortest paths between nodes in a graph. Our project extends this into a multi-objective label-setting variant.
Multi-objective Optimization	An area of multiple criteria decision making that is concerned with mathematical optimization problems involving more than one objective function to be optimized simultaneously.

C and Pointer Terms

Term	Definition
Pointer	A variable whose value is the direct memory address of another variable or data structure.
Dereferencing	The operation of accessing or manipulating the value stored in the memory location that a pointer points to, typically using the $*$ operator in C.
Memory Address	A unique identifier used by a device or CPU for data tracking. It dictates where specific data is stored within the RAM.
Stack Frame	A memory management technique used to store data and partial results, as well as to perform dynamic linking, return values for methods, and pass parameters for local variables during function execution.

References

[1] Julian Dibbelt, Thomas Pajor, and Tobias Zündorf. Multi-criteria routing in public transit networks. In *Symposium on Experimental Algorithms*, pages 1–14. Springer, 2015.

[2] Ernesto Queirós Vieira Martins. On a multicriteria shortest path problem. *European Journal of Operational Research*, 16(2):236–245, 1984.

[3] Matthias Müller-Hannemann and Mathias Schnee. Multicriteria shortest paths in time-dependent train networks. *Algorithms and Data Structures*, pages 34–46, 2010.